

**Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»**

Московский институт электроники и математики им. А.Н.Тихонова

Направление подготовки
«10.03.01 Информационная безопасность»
Образовательная программа **«Информационная безопасность»**

О Т Ч Е Т
о прохождении
производственной практики

Студент: Гутников Д.А.
(Фамилия И.О.)

БИБ233
номер группы

Руководитель практики студента:

Менеджер образовательных проектов ВК
должность и место работы

Глуховченко Е.В.
Фамилия И.О.

подпись

Руководитель практики от НИУ ВШЭ:

Зав. каф. информационной безопасности
киберфизических систем департамента
электронной инженерии МИЭМ НИУ ВШЭ
должность и место работы

Евсютин О.О.
Фамилия И.О.

подпись

Практика пройдена с оценкой _____

Дата _____

Москва, 2026

СОДЕРЖАНИЕ

1 Введение	3
2 Краткая характеристика организации	4
3 Описание профессиональных задач	5
4 Исполненное индивидуальное задание	6
4.1 Kubernetes.....	6
4.2 Kubernetes RBAC.....	7
4.3 Опасные права в Kubernetes RBAC	8
4.4 Реализация инструмента.....	11
4.5 Тестирование инструмента	14
5 Заключение.....	23
6 Список использованных источников.....	24

1 Введение

Целью прохождения производственной практики является получение профессиональных умений и опыта профессиональной деятельности, в первую очередь научно-исследовательской работы.

Задачи практики:

- закрепление и углубление теоретических знаний по прослушанным за время обучения в университете дисциплинам;
- адаптация студента к реальным условиям работы в различных учреждениях и организациях, приобретение опыта работы в трудовых коллективах, планирование работы в организации, коммуникация и общение в сфере будущей профессиональной деятельности;
- создание условий для практического применения знаний в области информационной безопасности;
- формирование и совершенствование базовых профессиональных навыков и умений в области применения современных информационных технологий к задачам информационной безопасности;
- выполнения обязанностей на первичных должностях в области обеспечения информационной безопасности;
- диагностика профессиональной пригодности студента к профессиональной деятельности;
- формирование информационной компетентности с целью успешной работы в профессиональной деятельности;
- обеспечение успеха дальнейшей профессиональной карьеры.

2 Краткая характеристика организации

VK – это одна из крупнейших IT-компаний России, в которой насчитывается более 200 продуктов, сервисов и приложений. Среди них социальные сети «ВКонтакте» и «Одноклассники», «Почта Mail.ru», сервисы «VK Клипы», «VK Музыка», «VK Видео» и другие.

История компании начиналась в 1998 году с создания почты и портала Mail.ru. Затем в 2007 году стала активно развиваться самостоятельная социальная сеть «ВКонтакте», которая в 2014 году перешла к Mail.ru Group. В 2021 году данная компания сделала ребрендинг и стала называться именем своего главного продукта – VK.

Сейчас VK является главным цифровым активом Российской Федерации и бесспорным лидером российского IT-сектора. VK Tech – самый быстрорастущий сегмент компании – увеличил выручку на 59% по итогам последнего отчётного периода. Другие сегменты компании, например, образовательные платформы (Учи.ру, Тетрика) и экосистемные сервисы (RuStore, Облако Mail.ru) также показали рост.

3 Описание профессиональных задач

Работа в рамках производственной практики делилась на теоретическую и практическую части.

Теоретическая часть:

- Изучение архитектуры Kubernetes;
- Анализ структуры RBAC: Roles, ClusterRoles, RoleBinding, ClusterRoleBinding;
- Анализ опасных прав в best practices и CIS Benchmark;
- Изучение wildcard-прав;

Практическая часть:

- Разработка архитектуры консольного инструмента;
- Реализация загрузки объектов Kubernetes через файлы-манифесты;
- Реализация возможности загрузки объектов через Kubernetes API;
- Реализация методов привязки субъектов и их прав в рамках ролей;
- Реализация фильтрации прав и их анализ на основе опасных правил, описанных в конфигурационном файле;
- Реализация методов итоговых отчётов в человекочитаемом и json форматах;
- Тестирование написанного приложения.

4 Исполненное индивидуальное задание

4.1 Kubernetes

Kubernetes – это оркестратор контейнерных приложений. Он используется для автоматизации развёртывания, масштабирования и координации в условиях кластера.

В Kubernetes ноды делятся на control-plane и worker. На control-plane располагаются объекты управления, а на worker-нодах запускаются пользовательские контейнеры.

Control-plane реализует последовательный механизм создания подов – наименьшую единицу в Kubernetes. Пайплайн создания контейнеров в рамках подов выглядит следующим образом:

1. API-сервер: AuthN → AuthZ → Admission → сохранение в etcd
2. Deployment controller: создание ReplicaSet
3. ReplicaSet controller: создание Pod в etcd
4. scheduler: назначение nodeName подам
5. kubelet (watch): обнаруживает Pod для своей ноды и запускает CRI
6. CRI: RunPodSandbox → PullImage → CreateContainer → StartContainer
7. containerd → runc → контейнер (процесс с PID)

Управление реализуется через API: все запросы поступают в API-сервер, где есть механизмы аутентификации и авторизации. Компрометация сервера (по порту 6443) ведёт к полному контролю над кластером.

Обработав и приняв запрос, API-сервер обращается к etcd, в котором фиксирует изменения в кластере. В etcd хранится информация о подах, приложениях, ролях, секретах и т.д. Доступ к незащищённому etcd по порту 2379 приводит к раскрытию конфиденциальной информации о кластере и к возможности её перезаписи.

Deployment Controller после изменения состояния etcd реализует ReplicaSet – объект Kubernetes, который фиксирует количество запускаемых в его рамках подов. Поды создаются не им, а контроллером ReplicaSet, который также фиксирует это в etcd.

Поды на данном этапе ещё не запущены, и чтобы их запустить, используется kubelet ноды, который определяет свои поды по ключевому слову nodeName. Этот параметр назначается scheduler'ом. Прямой доступ к kubelet (10250 порт) позволяет выполнять произвольные команды в подах в обход RBAC.

Kubelet запускает поды через Container Runtime Interface (CRI). Для запуска применяется containerd, включающий runc, который запускает контейнеры в подах. Перед созданием контейнера создаётся pause-контейнер, который нужен для захвата сетевого namespace. Это необходимо для стабильной работы и сохранения адресов, когда контейнер в поде перестаёт работать.

4.2 Kubernetes RBAC

Role-Based Access Control (RBAC) – встроенный в Kubernetes механизм авторизации, использующийся для разграничения прав доступа пользователей, групп и сервисных аккаунтов к ресурсам кластера.

После успешной аутентификации в API, запрос проходит через модуль авторизации, где принимается решение о нём по существующим в кластере правилам. Правила назначаются субъектам с помощью ролей:

- Role – роль субъекта в рамках своего пространства имён;
- ClusterRole – роль субъекта в рамках всего кластера;
- RoleBinding – объект Kubernetes, привязывающий конкретную роль к субъектам;
- ClusterRoleBinding – объект Kubernetes, привязывающий кластерную роль к субъектам;

Правила задаются следующими параметрами:

- Verbs – все действия, которые разрешено выполнять субъекту;
- Resources – все ресурсы, к которым применимы действия;
- ApiGroups – API-группы, к которым относятся ресурсы;

Пример такого правила представлен на рисунке 1. Данное правило означает, что субъекту будет доступна возможность получить подробную информацию о подах (get), вывести список всех доступных подов (list), просматривать все изменения подов в реальном времени (watch).

```
8  rules:
9    - apiGroups: [""]
10      resources: ["pods"]
11      verbs: ["get", "list", "watch"]
```

Рисунок 1 – Пример правила Kubernetes.

Модуль авторизации сопоставляет роль субъекта с запросом, который был отправлен. Если было найдено полное совпадение, то запрос пропускается, иначе запрос блокируется.

RBAC является одним из главных механизмов защиты и позволяет реализовать принцип минимальных привилегий (Zero-Trust). С другой стороны, избыточные права могут привести к повышению привилегий и получению контроля над кластером.

4.3 Опасные права в Kubernetes RBAC

Как было отмечено выше, безопасность кластера зависит от RBAC и от прав, которые выдаются субъектам. Выдача опасных прав может привести к повышению привилегий, обходу ограничений, получению доступа к конфиденциальной информации.

В данной работе опасными правами будут называться такие права Kubernetes, которые позволяют повысить изначальные привилегии или получить доступ к конфиденциальной информации о кластере.

Вопросам безопасности Kubernetes посвящено много методических материалов. Основными материалами для изучения являются «best practices» для Kubernetes и «CIS Kubernetes Benchmark». С их помощью были выбраны опасные правила для реализации статического анализатора прав ролевого контроля доступа.

В Kubernetes есть возможность указывать правила с помощью wildcard – доступ ко всему в рамках verbs, resources или apiGroups. Простое использование wildcard не делает правило опасным.

Описанные ниже права будут взяты из конфигурационного файла, который был создан для статического инструмента. По данным правам будет приведена краткая информация, что они дают и почему они опасны.

На рисунке 2 представлено правило, которое даёт возможность создавать или модифицировать уже существующие объекты, которые запускают контейнеры в кластере. Это правило опасно тем, что злоумышленник может создать произвольный контейнер в рамках данных объектов или изменить старые контейнеры, чтобы реализовать атаку.

```
- name: full_write_access
  rule:
    verbs: ["create", "patch", "update"]
    resources: ["pods", "deployments", "statefulsets", "daemonsets", "replicasets", "jobs", "cronjobs"]
    apiGroups: ["", "apps", "extensions", "batch"]
```

Рисунок 2 – Правило full_write_access.

На рисунке 3 представлено правило, предоставляющее возможность создавать процессы в поде (pods/exec) или присоединяться к уже существующим процессам (pods/attach). Предоставление доступа к данным ресурсам представляет угрозу безопасности, потому что данное правило даёт возможность компрометировать контейнеры внутри созданных подов и красть конфиденциальную информацию. С помощью полученных ресурсов злоумышленник сможет реализовать дальнейшую атаку на кластер.

```
- name: full_pod_shell
  rule:
    verbs: ["*"]
    resources: ["pods/exec", "pods/attach"]
    apiGroups: [""]
```

Рисунок 3 – Правило full_pod_shell.

На рисунке 4 представлено правило, которое будет давать доступ к конфиденциальным данным в подах: права на чтение, перечисление и просмотр изменений в реальном времени. Это правило создаёт угрозу конфиденциальности секретных данных.

```
- name: access_to_secrets
  rule:
    verbs: ["get", "list", "watch"]
    resources: ["secrets"]
    apiGroups: [""]
```

Рисунок 4 – Правило access_to_secrets.

На рисунке 5 показано правило, дающее возможность создавать JWT-токен в рамках пода. В рассматриваемых материалах по безопасности Kubernetes рекомендуется отключать автоматическое монтирование токена, чтобы исключить возможность его несанкционированного использования. Однако если у злоумышленника будет возможность создавать этот токен самостоятельно, защита будет обойдена и будет возможен обход аутентификации.

```
- name: create_sa_token_ability
  rule:
    verbs: ["create"]
    resources: ["serviceaccounts/token"]
    apiGroups: [""]
```

Рисунок 5 – Правило create_sa_token_ability.

На рисунке 6 приведено правило, предоставляющее права на любые действия в nodes/proxy. Данный объект используется для проксирования запросов в кластере, в том числе и к kubelet. Если злоумышленник получит доступ к nodes/proxy, то он сможет получить доступ и к kubelet, который будет выводить информацию о подах, контейнерах, состоянии узла.

```
- name: access_to_nodes_proxy
  rule:
    verbs: ["*"]
    resources: ["nodes/proxy"]
    apiGroups: ["*"]
```

Рисунок 6 – Правило access_to_nodes_proxy.

На рисунке 7 показано правило, которое позволяет создавать новые роли. Для злоумышленника это означает возможность реализовать эскалацию привилегий.

```
- name: create_role_ability
  rule:
    verbs: ["create"]
    resources: ["roles", "clusterroles"]
    apiGroups: ["rbac.authorization.k8s.io"]
```

Рисунок 7 – Правило create_role_ability.

На рисунке 8 представлено правило, позволяющее изменять права ролей, что также создаёт риск повышения привилегий.

```
- name: escalate_ability
  rule:
    verbs: ["escalate"]
    resources: ["roles", "clusterroles"]
    apiGroups: ["rbac.authorization.k8s.io"]
```

Рисунок 8 – Правило escalate_ability.

Правило на рисунке 9 позволяет привязывать роли с правами, которых у субъекта не было. Злоумышленник может привязать к себе права произвольной роли, приводящие к повышению привилегий.

```
- name: binding_ability
  rule:
    verbs: ["bind"]
    resources: ["rolebindings", "clusterrolebindings"]
    apiGroups: ["rbac.authorization.k8s.io"]
```

Рисунок 9 – Правило binding_ability.

Правило на рисунке 10 состоит в том, что обладатель этого права имеет возможность делать запросы от лица другого субъекта, который обладает необходимыми правами доступа. Например, злоумышленник с глаголом impersonate может сделать запрос от роли admin и, аналогично, повысить свои привилегии в кластере.

```
- name: impersonate_ability
  rule:
    verbs: ["impersonate"]
    resources: ["users", "groups", "serviceaccounts"]
    apiGroups: ["authentication.k8s.io"]
```

Рисунок 10 – Правило impersonate_ability.

Правило на рисунке 11 заключается в том, что субъект получает возможность модифицировать уже существующие пространства имён. Например, злоумышленник может модифицировать поле labels: pod-security.kubernetes.io/enforce и изменить значение restricted на privileged, что даст возможность использовать ранее недоступные функции.

```
- name: namespace_patch
  rule:
    verbs: ["patch"]
    resources: ["namespaces"]
    apiGroups: [""]
```

Рисунок 11 – Правило namespace_patch.

Данные правила охватывают основные сценарии повышения привилегий в Kubernetes и подходят для начального тестирования инструмента для проведения анализа

безопасности ролевого контроля доступа в Kubernetes. При дальнейшей работе с данным инструментом список правил может быть модифицирован.

4.4 Реализация инструмента

Программный инструмент был реализован с помощью языка python и включает в себя следующие функции:

- Интерфейс ввода консольной команды для запуска программы с параметрами;
- Загрузка файлов-манифестов из yaml-файлов или из Kubernetes API;
- Связывание ролей и субъектов;
- Оценка опасности правил на основе описанных в конфигурационном файле критериев. Конфигурационный файл написан в формате yaml и может быть дополнен новыми правилами;
- Вывод всех субъектов с их правами или только тех, правила которых были признаны опасными;
- Формирование итогового отчёта в человекочитаемом или json формате.

Конфигурационный файл представлен yaml-файлом. Минимальное содержимое данного файла представлено на рисунке 12. В данном файле есть заголовок `unsafe_rules`, который будет содержать список правил, содержащий поля «имя» и «правило». Правило задаётся полями «verbs», «resources» и «apiGroups».

```
configs > ! unsafe_rules.yaml
1 unsafe_rules:
2   - name: full_write_access
3     rule:
4       verbs: ["create", "patch", "update"]
5       resources: ["pods", "deployments", "statefulsets", "daemonsets", "replicasets", "jobs", "cronjobs"]
6       apiGroups: ["", "apps", "extensions", "batch"]
7
8   - name: full_pod_shell
9     rule:
10      verbs: ["*"]
11      resources: ["pods/exec", "pods/attach"]
12      apiGroups: [""]
13
14   - name: access_to_secrets
15     rule:
16       verbs: ["get", "list", "watch"]
17       resources: ["secrets"]
18       apiGroups: [""]
19
20   - name: create_sa_token_ability
21     rule:
22       verbs: ["create"]
23       resources: ["serviceaccounts/token"]
24       apiGroups: [""]
25
26   - name: access_to_nodes_proxy
27     rule:
28       verbs: ["*"]
```

Рисунок 12 – Конфигурационный файл небезопасных правил.

В рамках готового инструмента созданы следующие файлы с функциями:

- `kubernetes_functions.py`;
- `main.py`;
- `parser.py`;

- rbac_resolver.py;
- rule_analyzer.py;
- rule_deduplicator.py;
- saver.py;
- visual.py;
- yaml_analyzer.py.

В таблицах ниже опишем реализованные в данных файлах.

Таблица 1 – Функции kubernetes_functions.py.

Название функции	Параметры	Описание
load_kubernetes_manifests	Отсутствуют	Загружает манифесты из Kubernetes API
convert_roles	- manifests_dict – объект, содержащий, все загруженные из Kubernetes API манифесты	Конвертирует роли в формат, с которым будет работать реализованный инструмент
convert_bindings	- manifests_dict – объект, содержащий, все загруженные из Kubernetes API манифесты	Конвертирует привязки ролей в формат, с которым будет работать реализованный инструмент

Таблица 2 – Функции main.py.

Название функции	Параметры	Описание
build_parser	Отсутствуют	Создаёт парсер для CLI
analyze	- args – параметры запуска программы; - bindings – привязки ролей; - roles – роли	Анализирует роли и привязанные к ним субъекты по правилам, которые подгружаются из args
analyze_manifests	- args – параметры запуска программы	Загружает роли и привязки ролей из локальных манифестов и запускает функцию analyze
analyze_kubernetes	- args – параметры запуска программы	Загружает роли и привязки ролей из Kubernetes API и запускает функцию analyze
return_report	- args – параметры запуска программы; - result – результат анализа правил	Возвращает отчёт об анализе правил

Таблица 3 – Функции parser.py.

Название функции	Параметры	Описание
parse_bindings	- yaml_docs – подгруженные yaml-документы	Парсит привязки ролей
parse_binding	- yaml_doc – подгруженный yaml-документ	Вспомогательная функция для парсинга одной привязки роли
parse_subjects	-subjects_yaml – фрагмент binding, в котором содержится информация о субъектах привязки	Парсит фрагмент binding, содержащий информацию о субъектах
parse_roles	- yaml_docs – подгруженные yaml-документы	Парсит роли
parse_role	- yaml_doc – подгруженный yaml-документ	Вспомогательная функция для парсинга одной роли

Таблица 4 – Функции rbac_resolver.py.

Название функции	Параметры	Описание
get_role_rules	- binding – привязка ролей; - roles – роли, попавшие в функцию	Функция, которая определяет, какие роли попали в roleRefName binding'a
resolve_binding	- binding – привязка ролей; - roles – роли, попавшие в функцию	По результатам get_role_rules определяет субъекты, к которым применились правила и возвращает полученный результат
resolve_permissions	- bindings – привязки ролей, попавшие в функцию; - roles – роли, попавшие в функцию	Функция, которая проходит по всем привязкам и извлекает всех субъектов с выданными правами привязанных ролей

Таблица 5 – Функции rule_analyzer.py.

Название функции	Параметры	Описание
check_matches	- a – множество 1; - b – множество 2	Возвращает булево значение, есть ли пересечение в множествах (учитывается wildcard)
get_matches	- a – множество 1; - b – множество 2	Возвращает пересечение множеств (учитывается wildcard)
analyze_rule	- unsafe_rules – все права, которые были признаны небезопасными; - rule – текущее проверяемое правило	Возвращает все совпадения с опасными правилами в списке словарей: имя совпадения, глаголы, ресурсы и API-группы
check_rule	- unsafe_rules – все права, которые были признаны небезопасными; - rule – текущее проверяемое правило	Возвращает всю информацию по правилу, включая статистику по совпадениям с опасными правилами

Таблица 6 – Функции saver.py.

Название функции	Параметры	Описание
send_report	- args – параметры запуска программы; - report – полученный отчет в обработанном виде	В зависимости от параметров запуска либо сохраняет полученный результат, либо выводит его в консоль

Таблица 7 – Функции visual.py.

Название функции	Параметры	Описание
return_fast_result	- args – параметры запуска программы; - result – полученный отчет в сыром виде	Возвращает отчет в человекочитаемом формате
return_json_result	- args – параметры запуска программы; - result – полученный отчет в сыром виде	Возвращает отчет в json формате

Таблица 8 – Функции yaml_analyzer.py.

Название функции	Параметры	Описание
get_yaml	- filename – путь к файлу	Возвращает документ yaml в виде python-объекта
load_unsafe_rules	- unsafe_rule_file – путь к конфигурационному файлу с опасными правилами	Возвращает опасные правила в виде python-объекта
load_manifests	- paths – набор путей к файлам манифестам	Возвращает все файлы манифесты из найденных конфигурационных файлов

4.5 Тестирование инструмента

Для тестирования инструмента напишем файлы-манифесты, которые будут подаваться на вход данному инструменту: роль pod-reader (рисунок 13), привязка роли pod-reader (рисунок 14), кластерная роль node-reader (рисунок 15), привязка кластерной роли node-reader (рисунок 16).

```
test_manifests > ! Role.yaml
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: Role
3
4  metadata:
5    name: pod-reader
6    namespace: check
7
8  rules:
9    - apiGroups: [""]
10      resources: ["pods"]
11      verbs: ["get", "list", "watch"]
12
13    - apiGroups: [""]
14      resources: ["pods"]
15      verbs: ["create"]
```

Рисунок 13 – Пример роли.

```
test_manifests > ! ClusterRole.yaml
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: ClusterRole
3
4  metadata:
5    name: node-reader
6
7  rules:
8    - apiGroups: [""]
9      resources: ["nodes"]
10     verbs: ["get", "list", "watch"]
```

Рисунок 14 – Пример кластерной роли.

```

test_manifests > ! RoleBinding.yaml
 4  metadata:
 5    name: read-pods
 6    namespace: read-pods-namespace
 7
 8  subjects:
 9    - kind: User
10      name: john
11
12    - kind: ServiceAccount
13      name: name-sa
14      namespace: default
15
16  roleRef:
17    apiGroup: rbac.authorization.k8s.io
18    kind: Role
19    name: pod-reader

```

Рисунок 15 – Пример привязки роли.

```

test_manifests > ! ClusterRoleBinding.yaml
 1  apiVersion: rbac.authorization.k8s.io/v1
 2  kind: ClusterRoleBinding
 3
 4  metadata:
 5    name: admins
 6
 7  subjects:
 8    - kind: User
 9      name: admin
10
11  roleRef:
12    apiGroup: rbac.authorization.k8s.io
13    kind: ClusterRole
14    name: node-reader

```

Рисунок 16 – Пример привязки кластерной роли.

Стоит отметить, что все правила за исключением одного являются безопасными. Опасным правилом здесь является правило на создание подов, которое было записано в конфигурационном файле.

Запустим программу только с параметром пути манифестов и получим вывод программы. Для удобного восприятия выделим красным цветом часть с выводом опасных прав.

```

gutnikov@sr-gutnikov:~/kubernetes-rbac$ python3.11 main.py manifests --paths test_manifests
@safe: User:john
scope ---> read-pods-namespace
Role:pod-reader
apiGroups: ['']
resources: ['pods']
verbs: ['get', 'list', 'watch']

!not safe: User:john
scope ---> read-pods-namespace
Role:pod-reader
apiGroups: ['']
resources: ['pods']
verbs: ['create']
[{'unsafe_rule_name': 'full_write_access', 'matched_verbs': ['create'], 'matched_resources': ['pods'],
'matched_api_groups': ['']}]]

@safe: ServiceAccount:default:name-sa
scope ---> read-pods-namespace
Role:pod-reader
apiGroups: ['']
resources: ['pods']
verbs: ['get', 'list', 'watch']

!not safe: ServiceAccount:default:name-sa
scope ---> read-pods-namespace
Role:pod-reader
apiGroups: ['']
resources: ['pods']
verbs: ['create']
[{'unsafe_rule_name': 'full_write_access', 'matched_verbs': ['create'], 'matched_resources': ['pods'],
'matched_api_groups': ['']}]]

@safe: User:admin
scope ---> cluster
ClusterRole:node-reader
apiGroups: ['']
resources: ['nodes']
verbs: ['get', 'list', 'watch']

```

Как можно заметить, права на просмотр состояния подов и нод не были отмечены опасными, потому что их не было в конфигурационном файле опасных правил. С другой стороны, правило на создание подов было помечено флагом «!not safe». При выводе опасных правил также можно получить информацию о найденных пересечениях: имя совпавшего правила, совпавшие атрибуты этого правила.

Часто бывает необходимым вывести только опасные правила, именно поэтому в реализованном инструменте реализована функция вывода только опасных правил с помощью флага «--only-dangerous». В представленном выводе программы продемонстрировано, какие правила в рассматриваемых манифестах являются опасными и какие субъекты обладают данными правами.

```

gutnikov@sr-gutnikov:~/kubernetes-rbac$ python3.11 main.py manifests --paths test_manifests --only-
dangerous

```



```

!not safe: User:john
scope ---> read-pods-namespace
Role:pod-reader
apiGroups: [""]
resources: ['pods']
verbs: ['create']
[{'unsafe_rule_name': 'full_write_access', 'matched_verbs': ['create'], 'matched_resources': ['pods'],
'matched_api_groups': [""]}

!not safe: ServiceAccount:default:name-sa
scope ---> read-pods-namespace
Role:pod-reader
apiGroups: [""]
resources: ['pods']
verbs: ['create']
[{'unsafe_rule_name': 'full_write_access', 'matched_verbs': ['create'], 'matched_resources': ['pods'],
'matched_api_groups': [""]}

```

Реализованный инструмент также позволяет создавать итоговый отчёт в формате json. Для использования этой функции нужно использовать флаг «--json». Точно также выделим красным особо опасные правила.

```

gutnikov@sr-gutnikov:~/kubernetes-rbac$ python3.11 main.py manifests --paths test_manifests --json
{
  "User:john": [
    {
      "role": "pod-reader",
      "scope": "read-pods-namespace",
      "kind": "Role",
      "apiGroups": [
        ""
      ],
      "resources": [
        "pods"
      ],
      "verbs": [
        "get",
        "list",
        "watch"
      ],
      "matched": []
    },
    {
      "role": "pod-reader",
      "scope": "read-pods-namespace",
      "kind": "Role",
      "apiGroups": [
        ""
      ],
      "resources": [
        "pods"
      ],
      "verbs": [
        "create"
      ],
      "matched": [
        {

```

```

        "unsafe_rule_name": "full_write_access",
        "matched_verbs": [
            "create"
        ],
        "matched_resources": [
            "pods"
        ],
        "matched_api_groups": [
            ""
        ]
    }
}
],
"ServiceAccount:default:name-sa": [
    {
        "role": "pod-reader",
        "scope": "read-pods-namespace",
        "kind": "Role",
        "apiGroups": [
            ""
        ],
        "resources": [
            "pods"
        ],
        "verbs": [
            "get",
            "list",
            "watch"
        ],
        "matched": []
    },
    {
        "role": "pod-reader",
        "scope": "read-pods-namespace",
        "kind": "Role",
        "apiGroups": [
            ""
        ],
        "resources": [
            "pods"
        ],
        "verbs": [
            "create"
        ],
        "matched": [
            {
                "unsafe_rule_name": "full_write_access",
                "matched_verbs": [
                    "create"
                ],
                "matched_resources": [
                    "pods"
                ],
                "matched_api_groups": [
                    ""
                ]
            }
        ]
    }
]

```

```

    }
  ],
  "User:admin": [
    {
      "role": "node-reader",
      "scope": "cluster",
      "kind": "ClusterRole",
      "apiGroups": [
        ""
      ],
      "resources": [
        "nodes"
      ],
      "verbs": [
        "get",
        "list",
        "watch"
      ],
      "matched": []
    }
  ]
}

```

Аналогично можно вывести только опасные правила и сделать вывод короче:

```

gutnikov@sr-gutnikov:~/kubernetes-rbac$ python3.11 main.py manifests --paths test_manifests --json --only-dangerous
{
  "User:john": [
    {
      "role": "pod-reader",
      "scope": "read-pods-namespace",
      "kind": "Role",
      "apiGroups": [
        ""
      ],
      "resources": [
        "pods"
      ],
      "verbs": [
        "create"
      ],
      "matched": [
        {
          "unsafe_rule_name": "full_write_access",
          "matched_verbs": [
            "create"
          ],
          "matched_resources": [
            "pods"
          ],
          "matched_api_groups": [
            ""
          ]
        }
      ]
    }
  ],
  "ServiceAccount:default:name-sa": [

```

```

{
  "role": "pod-reader",
  "scope": "read-pods-namespace",
  "kind": "Role",
  "apiGroups": [
    ""
  ],
  "resources": [
    "pods"
  ],
  "verbs": [
    "create"
  ],
  "matched": [
    {
      "unsafe_rule_name": "full_write_access",
      "matched_verbs": [
        "create"
      ],
      "matched_resources": [
        "pods"
      ],
      "matched_api_groups": [
        ""
      ]
    }
  ]
}
}
}
}
}

```

Кроме работы с файлами в инструменте предусмотрено обращение к Kubernetes API для извлечения из запущенного кластера всех ролей и привязок. На рисунке 17 показано извлечение опасных правил, причём в консоли выделены написанные ранее произвольные роли, которые использовались для тестирования манифестной части инструмента.

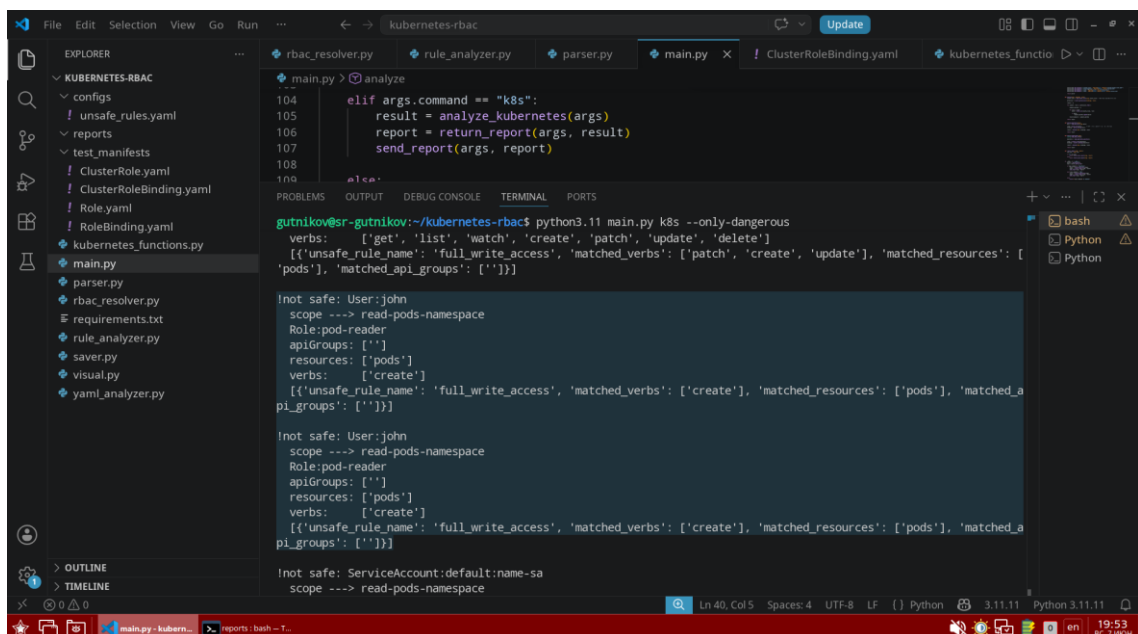


Рисунок 17 – Демонстрация работы инструмента.

Для тестирования работы инструмента с wildcard-правилами напишем роль, содержащую правила с «*» (рисунок 18). Для проверки будем использовать ресурсы «events» и «pods». Выдача всех глаголов ресурсу событий не создаст критическую угрозу безопасности повышения привилегий, а для подов данное правило будет особенно опасным. Для реализованной роли создадим привязку ролей, чтобы связать субъекты с ролью (рисунок 19).

```
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: Role
3
4  metadata:
5    name: wildcard-role
6    namespace: wildcard
7
8  rules:
9    - apiGroups: [""]
10      resources: ["events"]
11      verbs: ["*"]
12
13    - apiGroups: [""]
14      resources: ["pods"]
15      verbs: ["*"]
```

Рисунок 18 – Роль wildcard-role.

```
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: RoleBinding
3
4  metadata:
5    name: wildcard-binding
6    namespace: wildcard-binding
7
8  subjects:
9    - kind: User
10      name: john
11
12  roleRef:
13    apiGroup: rbac.authorization.k8s.io
14    kind: Role
15    name: wildcard-role
```

Рисунок 19 – Привязка роли wildcard-binding.

Запустим инструмент и оценим безопасность правил в написанной роли:

```

gutnikov@sr-gutnikov:~/kubernetes-rbac$ python3.11 main.py manifests --paths
test_manifests/wildcardRole.yaml test_manifests/wildcardRoleBinding.yaml
@safe: User:john
scope ---> wildcard-binding
Role:wildcard-role
apiGroups: ['']
resources: ['events']
verbs: ['*']

!not safe: User:john
scope ---> wildcard-binding
Role:wildcard-role
apiGroups: ['']
resources: ['pods']
verbs: ['*']
[{'unsafe_rule_name': 'full_write_access', 'matched_verbs': ['patch', 'update', 'create'], 'matched_resources':
['pods'], 'matched_api_groups': ['']}]]

```

Как можно заметить, правило с событиями было помечено как безопасное, в то время как второе правило, дающее все права при работе с подами было помечено как небезопасное. Это демонстрирует правильность реализации логики работы с wildcard.

5 Заключение

В рамках данной производственной практики был изучен ролевой контроль доступа в Kubernetes. Во время изучения литературы по информационной безопасности в системе Kubernetes был отобран список прав, которые могут приводить к повышению привилегий или раскрытию чувствительной информации.

В результате выполнения данной практической работы был реализован инструмент анализа прав RBAC в Kubernetes. Данный инструмент имеет функции, которые позволяют выводить список прав всех созданных субъектов или только те из них, которые являются опасными. Была реализована поддержка работы со статическими файлами и с Kubernetes API: роли можно подгружать из работающего кластера.

6 Список использованных источников

1. Kubernetes RBAC Good Practices [Электронный ресурс]. – URL: <https://kubernetes.io/docs/concepts/security/rbac-good-practices> (дата обращения: 07.06.2026).
2. Kubernetes Using RBAC Authorization [Электронный ресурс]. – URL: <https://kubernetes.io/docs/reference/access-authn-authz/rbac> (дата обращения: 07.06.2026).
3. CIS Kubernetes Benchmark [Электронный ресурс]. – URL: <https://www.cisecurity.org/cis-benchmarks> (дата обращения: 07.06.2026).
4. PyYAML Documentation [Электронный ресурс]. – URL: <https://pyyaml.org/wiki/PyYAMLDocumentation> (дата обращения: 07.06.2026).
5. Kubernetes-Python-Client Documentation (12 мая 2018 год). (дата обращения: 07.06.2026).